

BACSE344 - Cloud Infrastructure & Architecture

Comprehensive Database Engineering & Advanced SQL Report

Course: BACSE344 - Cloud Infrastructure & Architecture

Slot: C1 Slot

Faculty: Dr. P. Anandan

Project Title: CloudMed Healthcare Relational Database System

1. Database Creation, Tables & Integrity Constraints

The schema establishes strict relational integrity across 5 core entities: `departments`, `doctors`, `patients`, `appointments`, and `audit_logs`.

1.1 Database DDL Definition

```
CREATE DATABASE cloudmed_db WITH ENCODING = 'UTF8';
\c cloudmed_db;

-- 1. Departments
CREATE TABLE departments (
    dept_id SERIAL PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL UNIQUE,
    head_physician VARCHAR(100),
    building_block VARCHAR(20) NOT NULL,
    emergency_contact VARCHAR(20) NOT NULL
);

-- 2. Doctors
CREATE TABLE doctors (
    doctor_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    specialty VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL UNIQUE,
    dept_id INT NOT NULL REFERENCES departments(dept_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    consultation_fee NUMERIC(10, 2) NOT NULL DEFAULT 500.00 CHECK (consultation_fee ≥ 0),
    available_days VARCHAR(100) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

-- 3. Patients
CREATE TABLE patients (
    patient_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT NOT NULL CHECK (age BETWEEN 0 AND 125),
    gender VARCHAR(15) NOT NULL CHECK (gender IN ('Male', 'Female', 'Other')),
    blood_group VARCHAR(5) NOT NULL CHECK (blood_group IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-')),
    phone VARCHAR(20) NOT NULL UNIQUE,
    email VARCHAR(120) NOT NULL UNIQUE,
    address TEXT NOT NULL,
    medical_history TEXT
);

-- 4. Appointments
CREATE TABLE appointments (
    appointment_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL REFERENCES patients(patient_id) ON DELETE CASCADE,
    doctor_id INT NOT NULL REFERENCES doctors(doctor_id) ON DELETE RESTRICT,
    appointment_date DATE NOT NULL,
    appointment_time VARCHAR(20) NOT NULL,
    reason VARCHAR(255) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Scheduled' CHECK (status IN ('Scheduled', 'In-Progress', 'Completed', 'Cancelled')),
    notes TEXT,
    billing_amount NUMERIC(10, 2) NOT NULL DEFAULT 0.00 CHECK (billing_amount ≥ 0),
    is_paid BOOLEAN NOT NULL DEFAULT FALSE
);

-- 5. Audit Logs
CREATE TABLE audit_logs (
    log_id SERIAL PRIMARY KEY,
    entity_name VARCHAR(50) NOT NULL,
    operation_type VARCHAR(20) NOT NULL,
    entity_id INT NOT NULL,
    old_data JSONB,
    new_data JSONB,
    performed_by VARCHAR(50) DEFAULT CURRENT_USER,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

2. Real-World Datasets & Master Insertion

```
-- Departments & Doctors Seeding
INSERT INTO departments (dept_name, head_physician, building_block, emergency_contact) VALUES
('Cardiology', 'Dr. Sarah Jenkins', 'Block A - 3rd Floor', '+1-555-0101'),
('Neurology', 'Dr. Rajesh Patel', 'Block B - 2nd Floor', '+1-555-0102'),
('Pediatrics', 'Dr. Elena Rostova', 'Block C - 1st Floor', '+1-555-0103');

INSERT INTO doctors (name, specialty, email, phone, dept_id, consultation_fee, available_days) VALUES
('Dr. Sarah Jenkins', 'Interventional Cardiology', 'sarah.jenkins@cloudmed.io', '+1-555-0201', 1, 850.00, 'Mon, W'),
('Dr. Rajesh Patel', 'Clinical Neurology', 'rajesh.patel@cloudmed.io', '+1-555-0202', 2, 900.00, 'Tue, Thu, Sat'),
('Dr. Elena Rostova', 'Pediatric Medicine', 'elena.rostova@cloudmed.io', '+1-555-0203', 3, 600.00, 'Mon, Tue, Thu');

-- Patients & Appointments Seeding
INSERT INTO patients (name, age, gender, blood_group, phone, email, address, medical_history) VALUES
('Alice Johnson', 34, 'Female', 'O+', '+1-555-0301', 'alice.johnson@example.com', '742 Evergreen Terrace', 'Mild'),
('Robert Martinez', 52, 'Male', 'A+', '+1-555-0302', 'robert.martinez@example.com', '124 Conch Street', 'Type 2 D');

INSERT INTO appointments (patient_id, doctor_id, appointment_date, appointment_time, reason, status, billing_amount) VALUES
(1, 1, '2026-09-15', '09:30 AM', 'Annual Cardiac Checkup', 'Completed', 850.00, TRUE),
(2, 2, '2026-09-16', '02:00 PM', 'Chronic Migraine Assessment', 'Scheduled', 900.00, FALSE);
```

3. Relational Joins & Subqueries

3.1 Multi-Table INNER JOIN

```
SELECT a.appointment_id, p.name AS patient_name, d.name AS doctor_name, dept.dept_name, a.appointment_date, a.status
FROM appointments a
INNER JOIN patients p ON a.patient_id = p.patient_id
INNER JOIN doctors d ON a.doctor_id = d.doctor_id
INNER JOIN departments dept ON d.dept_id = dept.dept_id;
```

3.2 Correlated Subquery

```
SELECT p.name, a.appointment_date, a.reason
FROM patients p
JOIN appointments a ON p.patient_id = a.patient_id
WHERE a.appointment_date = (
    SELECT MAX(sub.appointment_date) FROM appointments sub WHERE sub.patient_id = p.patient_id
);
```

4. Views, Stored Procedures & Functions

4.1 Operational View (`vw_active_appointments`)

```
CREATE OR REPLACE VIEW vw_active_appointments AS
SELECT a.appointment_id, p.name AS patient_name, d.name AS doctor_name, a.appointment_date, a.appointment_time, a
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE a.status IN ('Scheduled', 'In-Progress');
```

4.2 Stored Procedure (`sp_book_appointment`)

```
CREATE OR REPLACE PROCEDURE sp_book_appointment(
    IN p_patient_id INT,
    IN p_doctor_id INT,
    IN p_date DATE,
    IN p_time VARCHAR(20),
    IN p_reason VARCHAR(255),
    OUT p_appt_id INT,
    OUT p_msg VARCHAR(255)
)
LANGUAGE plpgsql AS $$
DECLARE
    v_fee NUMERIC(10, 2);
BEGIN
    SELECT consultation_fee INTO v_fee FROM doctors WHERE doctor_id = p_doctor_id AND is_active = TRUE;
    IF NOT FOUND THEN
        p_msg := 'Error: Inactive or Invalid Doctor';
        RETURN;
    END IF;

    INSERT INTO appointments (patient_id, doctor_id, appointment_date, appointment_time, reason, status, billing)
    VALUES (p_patient_id, p_doctor_id, p_date, p_time, p_reason, 'Scheduled', v_fee)
    RETURNING appointment_id INTO p_appt_id;

    p_msg := 'Success: Booked appointment #' || p_appt_id;
END;
$$;
```

4.3 User-Defined Function (`fn_get_patient_risk_category`)

```
CREATE OR REPLACE FUNCTION fn_get_patient_risk_category(p_patient_id INT)
RETURNS VARCHAR(30) LANGUAGE plpgsql AS $$
DECLARE
    v_age INT;
    v_hist TEXT;
BEGIN
    SELECT age, COALESCE(medical_history, '') INTO v_age, v_hist FROM patients WHERE patient_id = p_patient_id;
    IF v_age ≥ 60 OR v_hist ILIKE '%cardiac%' THEN RETURN 'High Risk';
    ELSEIF v_age ≥ 40 OR v_hist ILIKE '%hypertension%' THEN RETURN 'Moderate Risk';
    ELSE RETURN 'Low Risk';
    END IF;
END;
$$;
```

5. Triggers, Cursors & ACID Transactions

5.1 Audit Trigger

```
CREATE OR REPLACE FUNCTION trg_fn_audit_appointments()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF (TG_OP = 'INSERT') THEN
        INSERT INTO audit_logs (entity_name, operation_type, entity_id, new_data) VALUES ('appointments', 'INSERT', TG_ARGV[0], TG_NEW);
    ELSIF (TG_OP = 'UPDATE') THEN
        INSERT INTO audit_logs (entity_name, operation_type, entity_id, old_data, new_data) VALUES ('appointments', 'UPDATE', TG_ARGV[0], TG_OLD, TG_NEW);
    ELSIF (TG_OP = 'DELETE') THEN
        INSERT INTO audit_logs (entity_name, operation_type, entity_id, old_data) VALUES ('appointments', 'DELETE', TG_ARGV[0], TG_OLD);
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_appointments_audit
AFTER INSERT OR UPDATE OR DELETE ON appointments
FOR EACH ROW EXECUTE FUNCTION trg_fn_audit_appointments();
```

5.2 Cursor Processing

```
CREATE OR REPLACE PROCEDURE sp_process_monthly_billing_batch(p_rate NUMERIC(4,2))
LANGUAGE plpgsql AS $$
DECLARE
    cur_unpaid CURSOR FOR SELECT appointment_id, billing_amount FROM appointments WHERE status = 'Completed' AND billing_status = 'Unpaid';
    v_rec RECORD;
BEGIN
    OPEN cur_unpaid;
    LOOP
        FETCH cur_unpaid INTO v_rec;
        EXIT WHEN NOT FOUND;
        UPDATE appointments SET billing_amount = v_rec.billing_amount * (1.0 - p_rate) WHERE appointment_id = v_rec.appointment_id;
    END LOOP;
    CLOSE cur_unpaid;
END;
$$;
```

5.3 ACID Transaction with Savepoint

```
BEGIN;
INSERT INTO patients (name, age, gender, blood_group, phone, email, address)
VALUES ('Mark Sterling', 40, 'Male', 'O+', '+1-555-0888', 'mark.sterling@cloudmed.io', '22 Silicon Way');
SAVEPOINT patient_created;

INSERT INTO appointments (patient_id, doctor_id, appointment_date, appointment_time, reason, status, billing_amount)
VALUES (currval('patients_patient_id_seq'), 1, '2026-09-22', '11:00 AM', 'Specialist Consultation', 'Scheduled', 0);

COMMIT;
```